

Ferry: Decoupling KV-State Lifetime from GPU Residency in Long-Context LLM Serving

Yi Zhong
yzhong76@ucsc.edu

Chen Yang
Institution B

Guanghua Sun
Institution C

Abstract

Modern LLM applications increasingly combine long contexts with external operations, such as retrieval, tool calls, and other services. Such requests unavoidably suspend while waiting for external results, creating a KV-residency dilemma. Keeping KV on GPU enables fast resume but ties up scarce memory, while discarding it frees memory but forces full re-prefill on resume. Host-memory swapping avoids recomputation, but a reactive swap still places KV transfer on the suspension path. To address this, we present *Ferry*, an LLM serving system that decouples KV-state lifetime from GPU residency through proactive checkpointing. *Ferry* incrementally checkpoints completed KV blocks to host memory while a request is running. This allows *Ferry* to take KV transfer off the suspension path: a request can release GPU-resident KV blocks immediately upon suspension, and later resume by reloading the checkpointed blocks and replaying only the uncheckpointed suffix. Our prototype implemented in vLLM shows that *Ferry* reduces resume latency by up to $5\times$ on tool-pause workloads, improves throughput by up to $1.23\times$ near saturation, and recovers $1.8\times$ faster from worker failure, with only 0.24% steady-state overhead.

1 Introduction

Large language models (LLMs) have advanced rapidly and now power a wide range of applications [2, 15, 21]. As context windows scale to hundreds of thousands of tokens, a single request can consume a significant fraction of GPU memory through its key-value (KV) cache, particularly for workloads such as long-form question answering [3] and document-level summarization [6]. Increasingly, such requests are no longer pure generation: they suspend mid-execution to invoke retrieval systems [11], external tools and APIs, or code-execution environments [23], and only resume once the external results return. These suspensions give rise to a *KV-residency dilemma*: retaining the KV cache on the GPU enables fast resumption but occupies scarce memory for the

duration of the wait, whereas evicting it reclaims memory but incurs a full re-prefill on resume, wasting compute and increasing end-to-end latency.

This pause-and-resume pattern, characterized as *interception* in prior work [1, 12, 13, 22], is the rule rather than the exception in modern LLM workloads such as retrieval-augmented generation [11] and agentic coding [23]. In the long-context regime, each suspension is fundamentally a KV-state management problem: the request is inactive from the GPU’s perspective, yet its already-computed KV cache must remain reachable for continuation and can occupy tens of gigabytes per request.

Existing serving systems address this only reactively, after a suspension has already occurred. Retaining the KV cache in GPU memory enables instant resumption but keeps scarce HBM occupied for the full duration of the external call. Discarding the cache and re-prefilling on resume frees memory but pays a large recomputation cost, which is particularly punishing in the long-context regime where prefill latency dominates end-to-end execution. A third option, *reactive swap*, moves the KV cache from GPU to host memory once the request suspends and reloads it when the external result returns [1, 12]; this avoids recomputation, but the transfer itself sits on the critical path — the request cannot release GPU memory until swap-out completes, and cannot resume until swap-in completes. All three strategies treat KV-state movement as a reaction to the suspension event, rather than something that can be prepared for in advance.

To address this, we present *Ferry*, an LLM serving system that decouples KV-state lifetime from GPU residency through *proactive checkpointing*. Our key observation is that the KV cache of a running request is constructed incrementally and is append-only at block granularity: once a KV block is completed, its contents do not change. *Ferry* exploits this by staging completed KV blocks to host memory in the background while the request is actively decoding, off the critical compute path. When the request later suspends, its GPU-resident KV blocks can be released immediately, because a host-side checkpoint already exists — no GPU-to-host

transfer is placed on the suspension path. On resume, *Ferry* replaces full re-prefill with a host-to-GPU reload of the checkpointed blocks followed by a short replay of the uncheckpointed suffix. KV-state movement is thereby shifted from the suspension critical path into the background of normal execution, while still avoiding recomputation.

We implement *Ferry* on top of vLLM [10] and evaluate it with Qwen2.5 models [18] on NVIDIA A6000 GPUs against GPU-resident retention, discard-and-reprefill, and reactive swap. *Ferry* reduces resume latency by up to $5\times$ on tool-pause workloads, sustains $1.17\text{--}1.23\times$ the throughput of the recompute baseline near saturation, and recovers $1.8\times$ faster from worker failure, while incurring only 0.24% steady-state overhead from continuous checkpointing.

2 Motivation and Background

Long-context LLM serving. LLM inference first processes the input prompt in the prefill phase and then generates output tokens one by one in the decode phase. Modern serving systems use iteration-level scheduling [24]. The KV cache produced during prefill and decode stays resident in GPU memory so later tokens can attend to previous context. vLLM [10] manages this KV cache as fixed-size pages, allowing concurrent requests to share the GPU KV pool efficiently. For long-context requests, however, the KV cache can occupy substantial HBM, making state management a first-order serving problem.

Augmented LLM workloads. Long-context LLM applications increasingly combine generation with retrieval, tools, and application-level control. These interactions interrupt normal decoding: a request may launch an external call and wait for the result before it can continue. Recent measurements of agent workloads report tool-induced idle gaps ranging from about 50 ms for local code execution to over 30 s for web and API calls [8]. Prior systems study this behavior from different angles, including interception handling [1], tool-call overlap [22], and program-level or tool-aware scheduling [12, 13]. **Reactive state handling.** Prior systems for augmented LLM serving decide what to do with KV state only after an external call blocks the request [1]. At that point, the live KV cache is still on the GPU, so freeing GPU memory requires either dropping the state or copying it out. For long-context requests, both choices are costly: dropping the state causes full prefill recomputation, while copying it out puts a large GPU-to-host transfer on the pause path. *Ferry* avoids this late reaction by publishing completed KV blocks while the request is still running.

GPU residency is not KV lifetime. Paged KV management and prefix caching improve how KV blocks are allocated and reused while they remain in the serving engine [10]. However, they do not make a request’s KV state independent of the GPU KV pool. Once the state is evicted, rerouted to another engine, or lost with a failed worker, the continuation must either find

another surviving copy or rebuild the state from the original prompt. Similarly, host offload and swap can move KV state out of GPU memory, but if the transfer starts only after the request blocks or memory pressure appears, it still lies on the critical path. This leaves a gap between making GPU KV management more efficient and making long-context continuation cheap after GPU residency is broken.

The cost of long-context state. For long prompts, losing GPU-resident KV state is expensive because the system must rebuild all KV produced before the intercept point. In our Qwen2.5-7B setup [18], prefilling four concurrent 16K-token requests takes about 10 s, and four 64K-token requests takes over 70 s. If an intercepted request discards its KV cache during an external wait, the system must repeat this work when the external result returns. If the system waits until the request pauses to move its KV cache to host memory, it avoids recomputation but still pays a burst of GPU-to-host transfer before the GPU memory can be released.

Key observation. KV state is produced incrementally as a request runs. At vLLM [10]’s block granularity, completed KV blocks are stable because later decoding appends new blocks rather than modifying old ones. Therefore, the system does not need to wait until a request pauses to preserve its state. It can copy completed blocks to host memory in the background and publish only complete blocks. When the request later pauses, most of its KV state has already been preserved. Suspension can then release GPU KV blocks, and resume can reload the latest complete checkpoint with bounded replay.

3 System Design

3.1 Overall Architecture

The overall architecture of *Ferry*, shown in Figure 1, centers on a checkpoint-reload substrate built on top of vLLM [10] that decouples the lifetime of a request’s KV state from its GPU residency. The substrate has three components: a) checkpoint store, b) reload path, and c) suspend-resume path. *Ferry* continuously publishes completed KV blocks to host memory while a request runs; on suspension, GPU KV blocks are released and reclaimed; on resume, the request reloads from the checkpoint and replays only the uncheckpointed suffix, avoiding full re-prefill. Because reload is independent of why the GPU blocks were released, the same substrate also supports cross-engine reroute and recovery from worker failure.

3.2 Host-side Checkpoint Store

The checkpoint store accumulates completed KV blocks in host memory in advance of any intercept point, moving GPU-to-host transfer off the suspension path. Reactive swap cannot release GPU memory until the live KV state has been copied to host; in contrast, completed blocks are already available before the intercept occurs, so suspension only records the

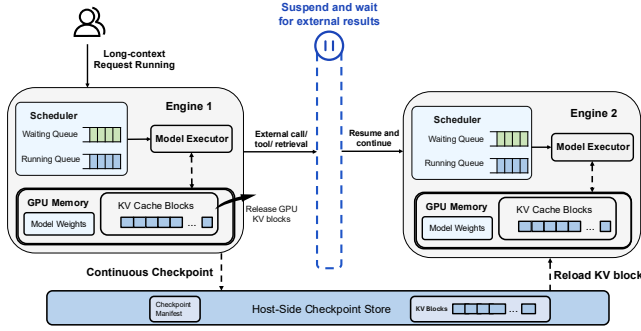


Figure 1: *Ferry*’s checkpoint-reload substrate for decoupling active KV state from GPU residency. A running request continuously checkpoints completed KV blocks to host memory; on external call, GPU KV blocks are released; on resume, the request reloads from the host store on the same or another engine.

latest checkpoint generation and releases the GPU KV blocks. Background transfer is practical because decode is HBM-bound and leaves PCIe largely idle on the critical path [10, 16, 26].

Ferry uses this otherwise idle GPU-to-host bandwidth to publish newly completed KV blocks while decoding continues. The high-level intuition mirrors VM pre-copy live migration [5] — move state before the switch is needed — but KV state is append-only at block granularity, so only newly completed blocks need to be published. Host RAM is the natural tier for this path: it offers much larger capacity than HBM while supporting reloads that are far cheaper than re-*prefill*. Peer-GPU staging depends on NVLink topology and consumes HBM on the target GPU, and SSD or remote storage adds read/write latency that would push suspension and restore work back onto the critical path.

Ferry checkpoints at the vLLM [10] block granularity, which matches the engine’s KV allocation unit and provides a natural atomic publish unit. Once a running request accumulates one complete KV block (16 tokens in our setup), the engine asynchronously copies its K and V tensors to the host store. A block is published only after both the GPU-to-host copy and the manifest update finish, so reload never uses a partially copied block. The manifest tracks the latest generation, the covered-token offset, and the logical-to-host block mapping needed to rebuild the request’s GPU KV layout during reload.

3.3 Checkpoint Reload and Bounded Replay

When a suspended request is selected for resume, the engine restores it from the latest complete checkpoint rather than re-*prefilling* the original prompt. It first reads the manifest to obtain the checkpoint generation, covered-token offset, and block mapping. It then allocates fresh GPU KV blocks,

copies the checkpointed K/V tensors back to GPU memory, and rebuilds the paged-attention layout.

Because checkpoints are published only at block boundaries, a resumed request lacks at most the tokens generated after the last published block. This turns replay into a bounded suffix repair rather than a second *prefill*: the engine reloads the checkpointed prefix, regenerates the uncovered suffix, and continues with any new tokens returned by the external call. Resume cost is therefore dominated by host-to-GPU KV reload plus a small bounded replay, not by reprocessing the original long prompt.

3.4 Suspend-Resume for Intercepted Requests

When an intercepted request issues an external call, its KV cache becomes idle until the result returns. *Ferry* suspends such a request from its latest published checkpoint, rather than retaining the KV blocks in GPU memory or discarding them outright.

Suspend decision. An intercepted request is suspended only when a checkpoint is available and keeping its idle GPU KV blocks would block waiting work; otherwise, the request keeps its KV blocks in GPU memory to avoid an unnecessary reload. Short waits favor retention because they avoid reload overhead, while longer waits favor suspension because the freed memory can serve other requests; Section 4 evaluates this tradeoff.

Suspend and resume. Suspending a request records its external-call state and latest checkpoint generation, removes it from the running set, and releases its GPU KV blocks. Only lightweight metadata remains: the request id, checkpoint generation, covered-token offset, and the application state needed after the external call. When the external result returns, the request re-enters the scheduler as resumable work, and once KV space is available, it resumes through the reload path of Section 3.3.

3.5 Generality Beyond Interception

The reload path is independent of why the GPU KV blocks were released. The same mechanism therefore generalizes to several serving events studied by prior work:

- **External pause.** The tool and retrieval pauses of Section 3.4 [1, 12, 13, 22], where a request waits for an external result.
- **Reroute.** A suspended request need not resume on the same engine [20]. It may return to its original engine when capacity becomes available, or migrate to another

engine for load balancing or recovery. The receiving engine allocates fresh GPU KV blocks, rebuilds the paged-attention layout from the checkpoint manifest, and continues from the checkpointed offset.

- **Worker failure.** If a worker loses its GPU KV blocks [14, 19], the request can still resume so long as the host checkpoint store remains available. A surviving worker reloads the latest complete generation and replays only the uncovered suffix, avoiding full prefill recomputation.

4 Evaluation

We evaluate *Ferry* to answer three questions: (1) what is the steady-state overhead of continuous checkpointing during normal decode, and how does checkpoint reload compare to full re-prefill in isolation? (§4.2) (2) on intercepted long-context workloads, how does *Ferry* compare to GPU-resident retention, discard-and-reprefill, and reactive swap in terms of resume latency, goodput, and memory occupancy? (§4.3) (3) does the same substrate enable resume across other events such as worker disruption, without paying full prefill recomputation? (§4.4)

4.1 Experimental Setup

Environment. We evaluate *Ferry* primarily on Qwen2.5-14B-Instruct [18] in fp16 with a maximum context length of 32K tokens. Unless otherwise stated, experiments run on two NVIDIA A6000 PCIe GPUs with 48 GB of memory each. For the reload microbenchmark in §4.2, we additionally use Qwen2.5-7B with long-context extension enabled to sweep context lengths up to 64K tokens.

Workloads. We use three workloads. The microbenchmark workload uses synthetic long-context prompts from 1K to 64K tokens to isolate the cost of re-prefill and checkpoint reload. The tool-pause workload uses arxivsumm [6] documents as long prompts. Each request decodes for a short period, issues a simulated external call, waits for a controlled pause duration, and then resumes after the external result is available. The disruption workload runs long-context requests on two engines and kills one engine during decoding to evaluate whether the affected requests can resume from host-side checkpoints instead of re-prefilling their original prompts.

Baselines. The evaluation compares *Ferry* with three resume strategies that do not use the checkpoint and reload substrate. The first is *recompute*, where unmodified vLLM [10] reprocesses the prompt on resume. The second is *GPU prefix caching*, where vLLM automatic prefix caching reuses KV only while it remains resident on the GPU. The third is *reroute with reprefill*, where a request is dispatched to another engine, but the receiving engine rebuilds the KV state by processing the prompt again instead of reloading KV.

Table 1: Steady-state decode overhead from continuous checkpointing on Qwen2.5-14B (16K prompt, 4096 output tokens, A6000). *Ferry* adds at most 0.79 ms (0.24 %) per token, even with 4 in-flight requests.

Workload	vLLM	+ checkpoint	Δ TPOT
Sequential (1 in-flight)	46.07 ms	46.07 ms	−0.00 ms
Concurrent (4 in-flight)	334.32 ms	335.11 ms	+0.79 ms (0.24 %)

4.2 Mechanism microbenchmarks

This section isolates the checkpoint and reload path. We measure the runtime overhead of continuous checkpointing during normal decoding, and then compare checkpoint reload with full prompt reprefill. These two measurements show whether *Ferry*’s basic mechanism is cheap enough for suspension and recovery.

Checkpoint overhead. Table 1 measures the cost of continuous checkpointing during normal decoding. The experiment uses Qwen2.5-14B [18] on a 16K-token prompt with 4k output tokens, and compares vLLM with *Ferry*’s background checkpoint writes.

For a single in-flight request, TPOT remains unchanged at 46.07 ms. With four concurrent long-context requests, TPOT increases from 334.32 ms to 335.11 ms, adding only 0.79 ms, a 0.24% relative overhead. The overhead remains small because the host transfer runs in the background while decoding continues on the GPU.

Reload versus re-prefill. Table 2 compares the wall-clock cost of restoring a request from a host-side checkpoint against running a full re-prefill, sweeping context length from 1K to 64K tokens on Qwen2.5-7B [18] on a single A6000 (batch 4, bf16). Reload mainly copies KV from host memory back to GPU memory, so its cost grows roughly linearly with KV size. The resulting speedup grows from $52\times$ at 1K to $86\times$ at 32K and $121\times$ at 64K. This is the structural property that lets *Ferry* treat preemption as a routine scheduling action: at long context, restoring a request from a checkpoint is nearly two orders of magnitude cheaper than restarting it from the prompt. For this sweep, we use Qwen2.5-7B so that a single A6000 can hold KV up to 64K context. The main end-to-end experiments use Qwen2.5-14B with 32K context because of GPU memory limits.

4.3 Tool Pause Resume

In augmented LLM serving, an interception pauses generation while the system waits for an external result, such as retrieval output or a tool response [1]. This experiment evaluates how different KV state policies affect resume latency after such a pause. A long-context request first decodes for a short period, then waits for a controlled pause duration to represent external latency, and finally resumes generation. We

Table 2: Checkpoint reload versus full re-prefill across context lengths on Qwen2.5-7B (A6000, batch 4). Reload is 52–121 $\times$ faster, with the gap widening at longer contexts.

Context	Re-prefill	Reload	Speedup
1K	0.49 s	9.4 ms	52 $\times$
2K	1.01 s	19.2 ms	53 $\times$
4K	2.09 s	38.4 ms	54 $\times$
8K	4.50 s	77.0 ms	58 $\times$
16K	10.40 s	153.7 ms	68 $\times$
32K	26.31 s	306.4 ms	86 $\times$
64K	73.83 s	611.8 ms	121 $\times$

measure resume latency from the end of the pause to the time the request continues producing tokens, comparing full recompute, automatic prefix caching, and *Ferry*’s reload from a host memory checkpoint.

Figure 2 reports resume latency under different pause durations. *Ferry* keeps resume latency low and stable, around 2.5–3 s across most of the sweep. Its checkpoint remains in host memory even when GPU memory is under pressure, so resume only requires reloading checkpointed KV state back to the GPU and replaying a bounded suffix.

APC is less stable because it depends on the KV state remaining resident on the GPU. Even at the shortest pause, APC takes about 5 s to resume. As the pause duration increases, the idle prefix is more likely to be evicted under memory pressure, and resume latency rises to about 9 s to 10 s. Full recompute is the slowest policy, taking about 10 s to 15 s because it reprefills the original long prompt on every resume.

Overall, *Ferry* has the lowest resume latency at every pause duration. The benefit is larger for longer pauses, where keeping KV in GPU memory becomes less reliable. The same effect appears in throughput. Near saturation, *Ferry* sustains 1.17 $\times$ to 1.23 $\times$ the completed throughput of the recompute baseline (Figure 3), because it avoids paying the full prefill cost after each pause.

Versus reactive swap. Reactive swap differs from *Ferry* mainly on the suspension path. In reactive swap, the system starts copying the paused request’s KV state from GPU memory to host memory only after an interception occurs, and GPU memory cannot be released until this copy finishes [1]. The resume step is similar to *Ferry*, since both paths reload KV from host memory back to GPU memory. However, reactive swap cannot release GPU memory immediately when a request pauses. It must first copy the request’s KV state from GPU memory to host memory, moving the same amount of KV data as the reload measured in Table 2. For long contexts, this adds hundreds of milliseconds to the pause path.

Ferry moves this copy earlier. Completed KV blocks are already published in the background while the request is running, so suspension only records the latest checkpoint gen-

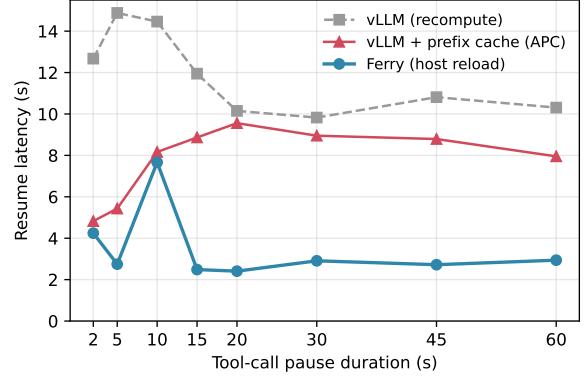


Figure 2: Resume latency versus tool-call pause duration on the tool-pause workload. *Ferry*’s host-checkpoint reload stays around 2.5–3 s regardless of pause length, while APC starts low but degrades to 8–9 s once the idle KV is evicted from GPU memory, and full recompute remains the slowest at 10–15 s.

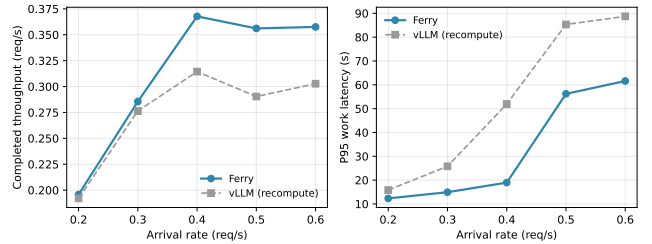


Figure 3: Completed throughput (left) and P95 work latency (right) versus arrival rate on the tool-pause workload. The gap between *Ferry* and the recompute baseline widens near saturation, since avoiding re-prefill on resume frees capacity that the baseline must spend on recomputation.

eration and releases the GPU KV blocks. The transfer cost therefore appears as the small steady overhead shown in Table 1, rather than as a burst on the suspension path. This difference becomes more important when multiple requests pause around the same time, because reactive swap creates synchronous GPU to host copies, while *Ferry* spreads the same data movement across normal decoding.

4.4 Disruption Recovery

When a GPU worker fails or restarts, its GPU-resident KV cache is lost. *Ferry* can recover because finalized checkpoint generations are kept in the host-side checkpoint store, whose lifetime is independent of the worker’s GPU KV pool.

We kill one of two engines mid-decode and measure the *failover gap* — the time from the kill until the affected request produces its next token on a surviving engine. Both *Ferry* and the *reroute-with-reprefill* baseline share the same failure-

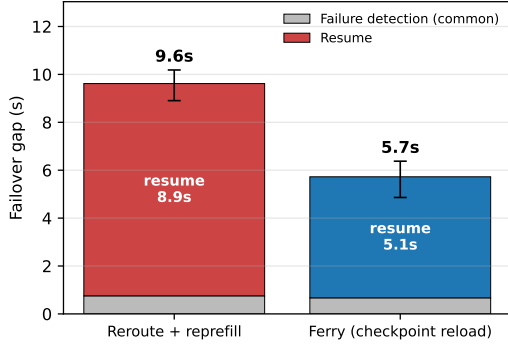


Figure 4: Failover gap after one of two engines is killed mid-decode, split into failure detection (common to both systems) and resume cost. *Ferry*’s checkpoint reload reduces the gap from 9.6 s to 5.7 s by avoiding re-prefill of the long prompt. Each bar reports the mean over 3 runs, with error bars showing the min/max across runs.

detection latency (router stale-status timeout plus redispach, ≈ 0.7 s), so the remaining difference is purely the resume cost on the receiving engine.

As shown in Figure 4, *Ferry* reduces the total failover gap from 9.6 s to 5.7 s. The baseline must re-prefill the long prompt on the receiving engine, paying 8.9 s of resume cost net of detection; *Ferry* instead reloads the request’s host checkpoint, paying only 5.0 s — a $1.8\times$ faster recovery on the resume path.

5 Discussion and Limitations

Ferry focuses on making continuation cheap after GPU KV blocks are released or lost. It does not require a new scheduling policy to show this benefit. Our prototype therefore uses a simple capacity-driven policy: a request is suspended or preempted only when waiting work would otherwise be blocked by KV memory.

This policy is enough for the workloads in this paper because *Ferry*’s main benefit comes from avoiding full re-prefill after suspension or failure. More aggressive SLO-aware scheduling [4, 7, 17, 25] can be built on top of the same reload path, but it is orthogonal to the checkpointing mechanism.

Our evaluation has three main limitations. First, it uses two engines, one hardware platform, and one model size. Scaling to larger clusters or heterogeneous GPUs would require coordinating multiple concurrent reroutes across surviving peers. Second, the disruption experiments assume that the host-side checkpoint store remains available after a worker failure. If the checkpoint store itself is lost, *Ferry* would need replication, which is outside the scope of this work. Third, *Ferry* assumes that the PCIe link is not saturated during decode. This holds in our setup, but may not hold in systems that already rely heavily on PCIe, such as prefill/decode dis-

aggregation [16, 26] or CPU offload [9]. In such settings, checkpoint traffic would need to be scheduled together with other PCIe transfers.

6 Conclusion

This paper studies how to manage active KV state in long-context LLM serving when requests suspend to wait for external operations such as tool calls or retrieval. We identify a *KV-residency dilemma*: retaining the KV cache on the GPU wastes scarce memory, while evicting it forces a full re-prefill on resume. We address this with *Ferry*, which continuously publishes completed KV blocks to host memory during execution. With this proactive checkpointing, a suspended request can release its GPU KV cache immediately and later resume through reload and bounded replay. The same substrate generalizes beyond interception to cross-engine reroute and recovery from worker failure. Our prototype on top of vLLM reduces resume latency by up to $5\times$ and improves throughput by up to $1.23\times$, while incurring only 0.24% steady-state overhead.

References

- [1] ABHYANKAR, R., HE, Z., SRIVATSA, V., ZHANG, H., AND ZHANG, Y. InferCept: Efficient intercept support for augmented large language model inference. *arXiv preprint arXiv:2402.01869* (2024).
- [2] ANTHROPIC. The Claude 3 model family: Opus, sonnet, haiku. <https://www.anthropic.com/news/claude-3-family>, 2024.
- [3] BAI, Y., LV, X., ZHANG, J., LYU, H., TANG, J., HUANG, Z., DU, Z., LIU, X., ZENG, A., HOU, L., ET AL. Longbench: A bilingual, multi-task benchmark for long context understanding. In *Proceedings of the 62nd annual meeting of the association for computational linguistics (volume 1: Long papers)* (2024), pp. 3119–3137.
- [4] CHEN, J., DU, C., LIU, R., YAO, S., YAN, D., LIAO, J., LIU, S., WU, F., AND CHEN, G. TokenFlow: Responsive LLM text streaming serving under request burst via preemptive scheduling. In *Proceedings of the 21st European Conference on Computer Systems* (2026), pp. 497–513.
- [5] CLARK, C., FRASER, K., HAND, S., HANSEN, J. G., JUL, E., LIMPACH, C., PRATT, I., AND WARFIELD, A. Live migration of virtual machines. In *Proceedings of the 2nd conference on Symposium on Networked Systems Design & Implementation-Volume 2* (2005), pp. 273–286.
- [6] COHAN, A., DERNONCOURT, F., KIM, D. S., BUI, T., KIM, S., CHANG, W., AND GOHARIAN, N. A discourse-aware attention model for abstractive summarization of long documents. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 2 (Short Papers)* (2018), pp. 615–621.
- [7] GOEL, K., MOHAN, J., KWATRA, N., ANUPINDI, R. S., AND RAMJEE, R. Niyama: Breaking the silos of LLM inference serving. *arXiv preprint arXiv:2503.22562* (2025).
- [8] GUO, D., WU, J., AND YIU, S. M. Saga: Workflow-atomic scheduling for ai agent inference on gpu clusters. *arXiv preprint arXiv:2605.00528* (2026).
- [9] JIANG, X., ZHOU, Y., CAO, S., STOICA, I., AND YU, M. NEO: Saving GPU memory crisis with CPU offloading for online LLM inference. *Proceedings of Machine Learning and Systems 7* (2025).

- [10] KWON, W., LI, Z., ZHUANG, S., SHENG, Y., ZHENG, L., YU, C. H., GONZALEZ, J., ZHANG, H., AND STOICA, I. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th symposium on operating systems principles* (2023), pp. 611–626.
- [11] LEWIS, P., PEREZ, E., PIKTUS, A., PETRONI, F., KARPUKHIN, V., GOYAL, N., KÜTTLER, H., LEWIS, M., YIH, W.-T., ROCKTÄSCHEL, T., ET AL. Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in neural information processing systems* 33 (2020), 9459–9474.
- [12] LI, H., HE, R., MANG, Q., ZHANG, Q., MAO, H., CHEN, X., ZHOU, H., CHEUNG, A., GONZALEZ, J., AND STOICA, I. Continuum: Efficient and robust multi-turn LLM agent scheduling with KV cache time-to-live. *arXiv preprint arXiv:2511.02230* (2025).
- [13] LUO, M., SHI, X., CAI, C., ZHANG, T., WONG, J., WANG, Y., WANG, C., HUANG, Y., CHEN, Z., GONZALEZ, J. E., ET AL. Autellix: An efficient serving engine for LLM agents as general programs. *arXiv preprint arXiv:2502.13965* (2025).
- [14] MIAO, X., SHI, C., DUAN, J., XI, X., LIN, D., CUI, B., AND JIA, Z. SpotServe: Serving generative large language models on preemptible instances. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2* (2024), pp. 1112–1127.
- [15] OPENAI. GPT-4 technical report. *arXiv preprint arXiv:2303.08774* (2023).
- [16] PATEL, P., CHOUKSE, E., ZHANG, C., SHAH, A., GOIRI, Í., MALEKI, S., AND BIANCHINI, R. Splitwise: Efficient generative LLM inference using phase splitting. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)* (2024), IEEE, pp. 118–132.
- [17] PATKE, A., REDDY, D., JHA, S., QIU, H., PINTO, C., NARAYANASWAMI, C., KALBARCZYK, Z., AND IYER, R. Queue management for SLO-oriented large language model serving. In *Proceedings of the 2024 ACM Symposium on Cloud Computing* (2024), pp. 18–35.
- [18] QWEN TEAM. Qwen2.5 technical report, 2025.
- [19] STRATI, F., MCALLISTER, S., PHANISHAYEE, A., TARNAWSKI, J., AND KLIMOVIC, A. Déjàvu: Kv-cache streaming for fast, fault-tolerant generative llm serving. *arXiv preprint arXiv:2403.01876* (2024).
- [20] SUN, B., HUANG, Z., ZHAO, H., XIAO, W., ZHANG, X., LI, Y., AND LIN, W. Llmunix: Dynamic scheduling for large language model serving. In *18th USENIX symposium on operating systems design and implementation (OSDI 24)* (2024), pp. 173–191.
- [21] TOUVRON, H., ET AL. LLaMA: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971* (2023).
- [22] XU, Y., KONG, X., CHEN, T., AND ZHUO, D. Conveyor: Efficient tool-aware LLM serving with tool partial execution. *arXiv preprint arXiv:2406.00059* (2024).
- [23] YANG, J., JIMENEZ, C. E., WETTIG, A., LIERET, K., YAO, S., NARASIMHAN, K., AND PRESS, O. Swe-agent: Agent-computer interfaces enable automated software engineering. *Advances in Neural Information Processing Systems* 37 (2024), 50528–50652.
- [24] YU, G.-I., JEONG, J. S., KIM, G.-W., KIM, S., AND CHUN, B.-G. Orca: A distributed serving system for Transformer-Based generative models. In *16th USENIX symposium on operating systems design and implementation (OSDI 22)* (2022), pp. 521–538.
- [25] ZHANG, W., WU, Z., MU, Y., NING, R., LIU, B., SARDA, N., LEE, M., AND LAI, F. JITServe: SLO-aware LLM serving with imprecise request information. In *23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI 26)* (2026), pp. 825–848.
- [26] ZHONG, Y., LIU, S., CHEN, J., HU, J., ZHU, Y., LIU, X., JIN, X., AND ZHANG, H. DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)* (Santa Clara, CA, 2024), USENIX Association, pp. 193–210.